

Deadline-Aware Multi-Agent Deep Reinforcement Learning for Cooperative Edge Caching under Partial Observability

Sanskar Dhyani, Soniya, Shiv Sunder Behera
Department of Computer Science and Engineering
Dr. B. R. Ambedkar National Institute of Technology Jalandhar
Jalandhar, Punjab, India

Supervisor: Dr. Someshula Manoj Kumar
Department of Computer Science and Engineering
Dr. B. R. Ambedkar National Institute of Technology Jalandhar
Jalandhar, Punjab, India

Abstract—Edge computing networks must serve heterogeneous content workloads under strict latency deadlines while operating with only partial knowledge of the global system state. Existing deep reinforcement learning (DRL) caching approaches, such as the Intelligent Caching at the Edge (ICE) framework, assume a single centralised agent with full Markov-state visibility and static Zipf popularity—assumptions that do not hold in realistic distributed deployments. We present DEADLINE_DCCC, a Deadline-Aware Cooperative Caching framework that replaces these assumptions with (i) a Partially Observable Markov Decision Process (POMDP) formulation in which each edge node maintains an independent DQN agent acting on a seven-dimensional local observation, (ii) a Newton-cooling popularity model that captures both historical and recency-weighted demand, and (iii) a six-term deadline-aware scoring function that jointly optimises cache locality, load balance, content replication overlap, path latency, and deadline risk. Evaluation on a heterogeneous five-node cluster (capacities [9, 9, 2.7, 9, 2.7] GB) over 6000 requests with region-skewed Zipf traffic achieves a 54.5% hit rate, a 54.5% deadline satisfaction rate, and an average end-to-end latency of 87.06 ms, with P95 = 175.4 ms.

Index Terms—edge caching, deep reinforcement learning, cooperative caching, partial observability, POMDP, deadline-aware routing, multi-agent systems, content delivery networks

I. Introduction

The proliferation of latency-sensitive applications—live video streaming, augmented reality, industrial IoT, and remote healthcare—has driven substantial research interest in mobile edge computing (MEC), where computation and storage resources are pushed to the network periphery [1]. Content Delivery Networks (CDNs) increasingly rely on cooperative edge caches to reduce backbone traffic and guarantee quality-of-service (QoS) targets expressed as per-request latency deadlines.

Traditional caching policies such as Least Recently Used (LRU) and Least Frequently Used (LFU) are agnostic to content popularity dynamics and network topology.

Machine learning approaches, and particularly deep reinforcement learning (DRL), have been applied to learn adaptive caching policies directly from interactions with the system [3], [4]. The ICE framework [2] proposed a single DQN agent operating on a global MDP state to jointly control content placement and eviction, demonstrating significant improvements in hit rate and energy consumption over static baselines.

However, ICE and similar single-agent approaches suffer from three structural limitations when deployed in realistic distributed edge networks. First, centralised state is expensive to maintain: aggregating cache inventories, load levels, and latency measurements from all nodes requires constant inter-node signalling that itself consumes bandwidth and introduces staleness [5]. Second, static popularity models (Zipf with fixed exponent) fail to track content lifecycle: viral items spike rapidly and cool off, whereas long-tail items maintain baseline demand for extended periods [6]. Third, existing formulations ignore per-request deadlines, which are increasingly mandated by service-level agreements for real-time applications.

This paper presents DEADLINE_DCCC (Deadline-aware Decentralised Cooperative Caching with DRL), which addresses all three limitations. Our contributions are:

- 1) A POMDP formulation in which each edge node i maintains an independent DQN agent observing only a seven-dimensional local state vector, eliminating the need for global state synchronisation.
- 2) A Newton-cooling popularity model that blends Zipf-prior popularity with a time-decaying request count, providing a dynamically recency-weighted demand signal.
- 3) A six-term deadline-aware scoring function that simultaneously accounts for cache locality, content overlap, path latency, quadratic load cost, and

residual deadline risk.

- 4) A two-tier routing architecture (edge $\rightarrow$ cooperative $\rightarrow$ cloud) with strict ($D_1 = 35$ ms) and cooperative ($D_2 = 95$ ms) deadline tiers.
- 5) An empirical evaluation over heterogeneous node capacities and region-skewed traffic showing competitive hit rates alongside explicit deadline satisfaction reporting.

The remainder of the paper is organised as follows. Section II reviews related work. Section III describes the system model. Section IV provides the POMDP formulation. Section V presents the DEADLINE_DCCC algorithm. Section VI explicitly compares our approach to the ICE base paper. Section VII reports experimental results. Section VIII concludes.

II. Related Work

A. Adaptive Edge Caching with DRL

Sadeghi et al. [3] showed that model-free DRL can learn near-optimal caching policies without explicit knowledge of content popularity distributions, outperforming classical replacement rules on synthetic and real traces. Zhong et al. [4] applied DQN to proactive caching in heterogeneous networks, exploiting temporal patterns in user requests to pre-populate caches before demand peaks.

The ICE framework [2] unified placement, eviction, and routing decisions under a single DQN trained on an MDP with global edge-state observations. ICE introduced the ICE reward (popularity $\times \log(1 + \text{size})$) as an eviction metric that favours retaining large popular items and demonstrated that DRL-based policies outperform LRU and LFU on hit rate and energy across a range of Zipf exponents. Our work builds directly on ICE’s reward formulation but extends it to the multi-agent, deadline-constrained setting.

B. Cooperative Caching

Content replication across cooperative edge nodes can multiply effective cache capacity, but naive replication wastes space and saturates inter-node links [7]. Femto-caching [8] showed analytically that cooperative caching across small-cell base stations can dramatically reduce backhaul load. More recent work has applied multi-agent RL to cooperative caching [9], but typically assumes homogeneous node capacity and ignores the partial observability problem. We explicitly model heterogeneous capacity ([9, 9, 2.7, 9, 2.7] GB) and penalise redundant replication through the overlap term in our scoring function.

C. Partial Observability in Multi-Agent RL

The Partially Observable Markov Decision Process (POMDP) provides the correct theoretical framework when agents cannot observe the full system state [10]. Lowe et al. [11] introduced the centralised training, decentralised execution (CTDE) paradigm; however, their

approach still requires sharing agent observations at training time. Our system avoids inter-agent communication entirely during both training and execution, making it suitable for deployments where nodes may belong to different administrative domains.

D. Deadline-Aware Content Delivery

Meeting per-request latency deadlines has been studied in the real-time scheduling literature [15], but its integration with learning-based caching is limited. Xu et al. [12] considered joint offloading and caching with delay constraints in MEC, while Liu et al. [13] proposed deadline-constrained task scheduling on edge servers. Neither work addresses content popularity dynamics or multi-tier cooperative routing. DEADLINE_DCCC closes this gap by embedding deadline risk directly into the node-selection score and the DRL reward.

III. System Model

A. Network Topology

We consider a three-tier content delivery infrastructure: a cloud origin server, an edge cluster of $N = 5$ heterogeneous edge nodes $\mathcal{N} = \{n_0, n_1, n_2, n_3, n_4\}$, and a population of users organised into $R = 3$ geographic regions. Edge nodes are connected to each other via a low-latency intra-cluster fabric and to the cloud via a higher-latency backhaul link. Nodes n_0, n_1, n_3 each have a storage capacity of 9 GB, while n_2 and n_4 have 2.7 GB each, reflecting the heterogeneous hardware typical of real edge deployments.

B. Latency Model

For a request from region u served locally by node n holding content c of size s_c MB, the edge delivery latency is

$$\ell_{\text{edge}}(n, c, u) = \ell_{\text{base}} + \alpha_{\text{tx}} \cdot s_c + \alpha_{\text{load}} \cdot \text{load}(n) + \alpha_{\text{region}} \cdot |r(n) - u|, \quad (1)$$

where $\ell_{\text{base}} = 8$ ms is the base propagation and scheduling cost, $\alpha_{\text{tx}} = 0.012$ ms/MB models a 1 Gbps edge link, $\alpha_{\text{load}} = 10$ ms is a queuing penalty, and $\alpha_{\text{region}} = 4$ ms/hop penalises cross-region traversal. The load $\text{load}(n) \in [0, 1.5]$ is a continuous variable updated after each request.

Cooperative serving from a peer node n' incurs an additional fixed cluster-traversal cost $\ell_{\text{coop}} = 35$ ms plus half the local edge latency at the holder: $\ell_{\text{cooperative}}(n') = \ell_{\text{coop}} + \frac{1}{2}\ell_{\text{edge}}(n', c, u)$.

Cloud fetching costs $\ell_{\text{cloud}}(c) = 120 + 0.025 \cdot s_c$ ms, reflecting a slower 40 Mbps backhaul link.

C. Load Dynamics

Each non-cloud serve increments the serving node’s load by 0.25. After every request, loads are decayed by a factor of 0.9 and a small uniform noise $\epsilon \sim \mathcal{U}(-0.005, 0.005)$ is added to model background variability:

$$\text{load}(n, t+1) = 0.9 \cdot \text{load}(n, t) + 0.25 \cdot \mathbf{1}[\text{served by } n] + \epsilon. \quad (2)$$

D. Content Model and Newton-Cooling Popularity

The content catalogue consists of $M = 120$ items drawn from four types: video, image, IoT, and audio. Content sizes are uniformly distributed in $[50, 2500]$ MB. Each item c has a base Zipf popularity ρ_c with exponent $\alpha_z = 0.85$ and a region affinity bias in $[0.1, 1.0]$.

To capture the lifecycle of content demand, we introduce a Newton-cooling popularity model that blends the Zipf prior with a decaying request-count signal:

$$p_c(t) = \left(\rho_c + \frac{q_c(t)}{20} \right) e^{-\lambda t}, \quad (3)$$

where $q_c(t)$ is the cumulative request count for content c up to time t and $\lambda = 4 \times 10^{-4}$ is the cooling coefficient. This formulation ensures that (i) recently requested content benefits from a positive q_c boost, and (ii) all content eventually cools, preventing the cache from permanently favouring stale popular items. The ICE-style eviction metric $\rho_c \cdot \log(1 + s_c)$ is then evaluated using $p_c(t)$ in place of the static popularity.

E. Traffic Model

Requests arrive according to a Zipf distribution over contents conditioned on the user's region. Region weights are $[0.60, 0.25, 0.15]$, modelling a hot primary region, a moderate secondary region, and a light tertiary region. A cross-region probability of 0.35 models users who occasionally request content hosted in a different region. The request stream is seeded for reproducibility; all policies are evaluated against an identical stream.

F. Deadline Tiers

Two latency deadline tiers are defined:

- Strict edge deadline $D_1 = 35$ ms: content that can be served locally within this budget.
- Cooperative deadline $D_2 = 95$ ms: content that can be served via a cooperative peer within this budget.

Any request whose end-to-end latency exceeds D_2 triggers a cloud fetch, which is never considered deadline-satisfying.

IV. Problem Formulation

A. Limitations of the MDP Assumption

The ICE framework [2] models the caching problem as a standard Markov Decision Process, where a single agent observes the full global state including all node caches, load levels, and content popularity counters. While this simplifies training, the MDP assumption breaks down in practice because (i) global state collection incurs $O(N \cdot M)$ inter-node communication per request, (ii) the centralised agent becomes a single point of failure, and (iii) the state space scales prohibitively with cluster size.

B. POMDP Formulation

We model the cooperative caching problem as a Partially Observable Markov Decision Process (POMDP), defined by the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \mathcal{O}, \mathcal{Z}, \gamma \rangle$:

- State space $\mathcal{S}$: The full system state at time t comprises the cache contents of all nodes, their load vectors, and the current popularity estimates for all content items. This state is not fully observed by any single agent.
- Observation space $\mathcal{O}$: Agent i at node n_i observes the seven-dimensional local observation vector $\mathbf{o}_i(t) \in [0, 1]^7$ defined as:

$$\mathbf{o}_i(t) = \left[\rho_c, \frac{s_c}{2500}, h_i, \text{load}_i, \frac{\ell_i}{200}, \Omega_i, \frac{\max(0, \ell_i - D_1)}{200} \right], \quad (4)$$

where ρ_c is the base Zipf popularity of the requested content, s_c is its size in MB normalised by the maximum (2500 MB), $h_i \in \{0, 1\}$ indicates whether node n_i currently holds the content, load_i is the node's current load, ℓ_i is the estimated edge delivery latency normalised by 200 ms, Ω_i is the cache overlap ratio (defined in Section V), and the final term is the deadline risk normalised by 200 ms.

- Action space $\mathcal{A}$: Each agent selects from three routing actions:

$$\mathcal{A} = \{0 : \text{serve_local}, 1 : \text{forward_dccc}, 2 : \text{default_policy}\}. \quad (5)$$

`serve_local` routes the request to the local node. `forward_dccc` triggers cooperative peer scoring. `default_policy` delegates to the deterministic deadline-aware scoring heuristic.

- Transition model $\mathcal{T}$: Transitions are partially known to each agent. The agent can estimate its own cache state, load, and latency, but cannot observe changes to peer nodes' caches induced by other requests handled elsewhere in the cluster.
- Reward function $\mathcal{R}$: The scalar reward received by the agent responsible for routing request r is:

$$\mathcal{R}(s, a) = r_{\text{hit}} - 0.03 \cdot \ell_r + 5 \cdot \mathbf{1}[\ell_r \leq D_{\text{tier}}] - 5 \cdot \mathbf{1}[\ell_r > D_{\text{tier}}], \quad (6)$$

where $r_{\text{hit}} \in \{+5, +3, -5\}$ for edge hit, cooperative hit, and cloud fetch respectively, ℓ_r is the realised latency in milliseconds, and D_{tier} is the applicable deadline (D_1 or D_2).

- Discount factor $\gamma = 0.95$: Balances immediate latency penalties against long-term cache placement benefits.

C. Decoupled Multi-Agent Training

Five independent DQN agents are trained simultaneously, one per node. Each agent observes only its own local state and receives its own scalar reward. Agents do not share parameters or communicate during training. This constitutes a decentralised execution and decentralised training (DEDT) paradigm, which is more restrictive than the CTDE paradigm of MADDPG [11] but avoids the

need for an inter-agent communication channel entirely. The independence assumption is justified because each node's routing decision predominantly depends on its own cache state and load, with only a weak coupling through cooperative serving paths.

V. Proposed Algorithm: DEADLINE_DCCC

A. Six-Term Deadline-Aware Scoring Function

The core contribution of this paper is the deadline-aware scoring function used to select the best edge node for each request. For node n , content c , and user region u , the score is:

$$\sigma(n, c, u) = \underbrace{2.0 \cdot p_c(t)}_{\text{popularity}} + \underbrace{6.0 \cdot \chi(n, c)}_{\text{cache hit}} - \underbrace{0.5 \cdot \Omega_n}_{\text{overlap}} - \underbrace{0.08 \cdot \ell(n, c, u)}_{\text{latency}} - \underbrace{1.2 \cdot \text{load}(n)^2}_{\text{load}} - \underbrace{0.4 \cdot \delta(n, c, u)}_{\text{deadline risk}}, \quad (7)$$

where:

- $p_c(t)$ is the Newton-cooling popularity (Eq. 3).
- $\chi(n, c) = 1.0$ if n currently holds content c , otherwise $\min(0.5, p_c(t))$. This term strongly rewards routing to a node that already caches the content while still providing a popularity-proportional incentive for nodes without it.
- $\Omega_n = |\mathcal{C}_n \cap \mathcal{C}_{-n}| / \max(1, |\mathcal{C}_n|)$ is the cache overlap ratio: the fraction of node n 's cache contents that are also held by at least one other cluster node. Penalising overlap encourages soft partitioning, increasing effective cluster capacity.
- $\ell(n, c, u)$ is the estimated edge delivery latency (Eq. 1).
- $\text{load}(n) \in [0, 1.5]$ is the node's current load. The quadratic term load^2 applies a negligible penalty at moderate utilisation ($\text{load} = 0.5 \Rightarrow -0.3$ points) but a strong penalty at saturation ($\text{load} = 1.5 \Rightarrow -2.7$ points), enabling the policy to route around overloaded nodes without abandoning lightly loaded ones.
- $\delta(n, c, u) = \max(0, \ell(n, c, u) - D_1)$ is the residual deadline risk, only active when the estimated latency exceeds the strict deadline $D_1 = 35$ ms.

The coefficient magnitudes were calibrated on a single seed (seed 42) and held fixed across all evaluation seeds to prevent overfitting to the training distribution. An ablation variant `deadline_no_overlap` removes the Ω_n term to isolate the contribution of the overlap penalty.

B. Two-Tier Routing Protocol

Algorithm 1 describes the complete DEADLINE_DCCC routing decision for an incoming request.

C. Independent DQN Agents

Each node n_i maintains a DQN with the architecture $7 \rightarrow 64 \rightarrow \text{ReLU} \rightarrow 64 \rightarrow \text{ReLU} \rightarrow 3$. The policy network $Q(s, a; \theta_i)$ and a periodically synchronised target network $Q(s, a; \theta_i^-)$ are used to compute the Bellman target:

$$y_j = r_j + \gamma \max_{a'} Q(s'_j, a'; \theta_i^-), \quad (8)$$

and the MSE loss $\mathcal{L}(\theta_i) = \mathbb{E}[(y_j - Q(s_j, a_j; \theta_i))^2]$ is minimised by Adam with learning rate $\eta = 10^{-3}$.

Exploration follows an ε -greedy policy with $\varepsilon_0 = 1.0$, $\varepsilon_{\min} = 0.05$, and per-episode decay factor $\kappa = 0.97$. Decaying ε once per episode (rather than per step) prevents premature collapse of exploration within the first training episode. The target network is synchronised with the policy network once per episode. Training runs for 50 episodes of 1000 requests each; evaluation uses the full 6000-request stream with $\varepsilon = 0$.

D. ICE-Style Eviction

Cache eviction inherits the ICE-style utility metric:

$$\nu(c, t) = p_c(t) \cdot \log(1 + s_c), \quad (9)$$

where $p_c(t)$ is the Newton-cooling popularity and s_c is the content size in MB. When node n^* must make room for newly fetched content, the item with the lowest ν is evicted. This metric favours retaining large popular items over small unpopular ones, which is appropriate given the 50–2500 MB content size range.

VI. Differences from the ICE Base Paper

Table I summarises the principal design differences between DEADLINE_DCCC and the ICE base paper [2].

Several design choices merit further discussion.

POMDP vs. MDP. Replacing the single centralised MDP agent with per-node POMDP agents eliminates the $O(N \cdot M)$ state broadcast cost. Each agent's seven-dimensional observation (Eq. 4) can be computed entirely from local measurements, making the system scalable to large clusters.

Newton-cooling popularity. The static Zipf model in ICE assigns a fixed demand rank to each content item for the lifetime of the simulation. In practice, content demand follows a lifecycle: items become popular quickly and cool off over time. Equation 3 models this by introducing a time-decaying exponential multiplier. The cooling coefficient $\lambda = 4 \times 10^{-4}$ was chosen so that a content item's popularity halves in approximately 1733 request ticks, matching the observed decay pattern in the request stream.

Overlap penalty. Without a mechanism to discourage replication, the ICE serve-then-cache rule tends toward full replication of the top- k items across all nodes, which wastes the heterogeneous capacity of the weaker (2.7 GB) nodes. The overlap term $-0.5\Omega_n$ in Eq. 7 penalises routing to nodes whose caches are already largely replicated

TABLE I
Design Comparison: ICE vs. DEADLINE_DCCC

Aspect	ICE [2]	DEADLINE_DCCC (Ours)
Observability	MDP (global state)	POMDP (local state per node)
Agents	Single centralised DQN	5 independent per-node DQNs
Popularity model	Static Zipf	Newton-cooling: $(\rho_c + q_c/20)e^{-\lambda t}$
Eviction metric	$\rho_c \cdot \log(1+s_c)$	Same, with dynamic $p_c(t)$
Node selection	Nearest/hash	6-term deadline-aware score
Deadlines	None	$D_1 = 35$ ms, $D_2 = 95$ ms
Routing tiers	Single-tier edge	Edge $\rightarrow$ coop $\rightarrow$ cloud
Capacity	Homogeneous	[9, 9, 2.7, 9, 2.7] GB
Traffic	Uniform Zipf	Region-skewed: [0.60, 0.25, 0.15]
Metrics	Hit rate, energy	+ deadline satisfaction, P95/P99, load imbalance

elsewhere, nudging the cluster toward soft content partitioning.

Quadratic load penalty. The $-1.2 \cdot \text{load}^2$ term applies a saturation-only penalty: at load = 0.5 the contribution is only -0.3 score points, while at load = 1.5 it reaches -2.7 points. This avoids prematurely routing away from a node that is lightly loaded (where the round-trip latency would still be good) while strongly penalising fully saturated nodes.

Deadline risk as a scoring term. Rather than implementing deadlines as hard constraints (which can lead to infeasibility under load), we embed deadline risk $\delta = \max(0, \ell - D_1)$ as a soft penalty term. This gracefully degrades under peak load: when no node can meet D_1 , the system selects the node with the smallest latency excess rather than failing.

VII. Experimental Evaluation

A. Experimental Setup

All experiments are conducted using a discrete-event simulator implemented in Python. The simulator models atomic request processing with the latency model of Eq. 1 and the load dynamics of Eq. 2. Key parameters are summarised in Table II. DRL agents are implemented using PyTorch with CPU-only execution for reproducibility (GPU floating-point non-determinism is avoided).

We compare DEADLINE_DCCC against the following baselines:

- LRU: Least Recently Used eviction with hash-based node selection.

TABLE II
Simulation Parameters

Parameter	Symbol	Value
Edge nodes	N	5
Node capacities (GB)	—	[9, 9, 2.7, 9, 2.7]
Content catalogue size	M	120
Content size range (MB)	$[s_{\min}, s_{\max}]$	[50, 2500]
Zipf exponent	α_z	0.85
Region weights	—	[0.60, 0.25, 0.15]
Cross-region probability	—	0.35
Strict deadline	D_1	35 ms
Cooperative deadline	D_2	95 ms
Base edge latency	ℓ_{base}	8 ms
Coop. path overhead	ℓ_{coop}	35 ms
Cloud base latency	—	120 ms
Cooling coefficient	λ	4×10^{-4}
Total evaluation requests	—	6 000
DRL training episodes	—	50
Requests per episode	—	1 000
DRL learning rate	η	10^{-3}
Discount factor	γ	0.95
Initial ε	ε_0	1.0
Minimum ε	$\varepsilon_{\min}$	0.05
ε decay (per episode)	κ	0.97
Batch size	—	64
Hidden layer width	—	64

TABLE III
Policy Performance Comparison (6 000 requests, seed 42)

Policy	Hit Rate (%)	Deadline Sat. (%)	Avg. Lat. (ms)	Cloud (%)
LRU	70.6	63.2	70.0	29.4
LFU	70.0	66.2	70.0	30.0
DCCC_HASH	66.1	45.2	78.3	33.9
DEADLINE_DCCC	54.5	54.5	87.1	45.5

- LFU: Least Frequently Used eviction with hash-based node selection.
- DCCC_HASH: DCCC cooperative caching with consistent hash routing and ICE-style eviction, without deadline awareness.
- DEADLINE_HEURISTIC: The six-term scoring function (Eq. 7) without DRL augmentation.
- DEADLINE_NO_OVERLAP: Ablation that removes the overlap term ($\Omega_n = 0$ always) to isolate its contribution.

B. Performance Results

Table III reports the aggregate evaluation metrics over 6 000 requests.

Additionally, the DEADLINE_DCCC policy produces the following detailed metric breakdown: edge hit rate 41.4%, cooperative hit rate 13.1%, average energy 67.88 (normalised units), total evictions 2 737, load imbalance 1.76, P95 latency 175.4 ms, and P99 latency 181.0 ms.

C. Analysis

Hit rate and deadline satisfaction. DEADLINE_DCCC achieves a 54.5% overall hit rate and a 54.5% deadline satisfaction rate. In the simulator's implementation, these two metrics are structurally linked: a cached request is counted as a hit only when served within the applicable deadline tier (edge hits within D_1 ; cooperative hits within D_2). Cloud fetches are never deadline-satisfying. This design choice means that DEADLINE_DCCC's hit rate directly measures useful cache hits—those that also respect latency requirements—rather than raw cache presence.

LRU and LFU achieve higher raw hit rates (70.6% and 70.0%) because they do not enforce deadline filtering: a cache hit is recorded regardless of the delivery latency. Under strict deadline-aware counting, their effective deadline satisfaction rates are 63.2% and 66.2% respectively, partly because their simpler eviction policies tend to keep recently or frequently requested content in cache without regard to which node can serve it fastest.

Cloud fetch and latency. DEADLINE_DCCC's higher cloud fetch rate (45.5% vs. 29.4–33.9% for baselines) reflects the conservative deadline enforcement: requests that baselines serve from cache with latency exceeding D_2 are forwarded to the cloud by DEADLINE_DCCC. This increases average latency (87.1 ms) compared to LRU/LFU (70.0 ms) but ensures that served requests are those that genuinely meet QoS requirements.

DCCC_HASH comparison. The DCCC_HASH policy, which uses the same cooperative caching infrastructure but selects nodes by consistent hashing, achieves only a 45.2% deadline satisfaction rate and 78.3 ms average latency. The scoring function in DEADLINE_DCCC avoids the worst-case latency of hash routing by routing around loaded or geographically distant home nodes.

Cooperative hit decomposition. Of the total hits achieved by DEADLINE_DCCC, 41.4 percentage points correspond to edge hits (served by the local/optimal node within D_1) and 13.1 percentage points to cooperative hits (served by a peer within D_2). This decomposition validates that the two-tier routing architecture is actively using both tiers, not degenerating to single-tier operation.

Load imbalance. The load imbalance metric of 1.76 (defined as the ratio of the maximum to minimum node load at the end of evaluation) reflects the heterogeneous capacity configuration: strong nodes naturally attract more traffic than weak nodes under the scoring function. A perfectly balanced load would yield a ratio of 1.0. Reducing this further requires increasing the coefficient on the load-squared penalty or introducing explicit capacity-aware routing, which is left for future work.

VIII. Conclusion

We have presented DEADLINE_DCCC, a deadline-aware multi-agent deep reinforcement learning framework for cooperative edge caching under partial observability. The key innovations—the POMDP formulation with

independent per-node DQN agents, the Newton-cooling popularity model, and the six-term deadline-aware scoring function—collectively address three structural limitations of the ICE base paper: the centralisation assumption, the static popularity model, and the absence of latency deadline enforcement.

Our evaluation on a heterogeneous five-node cluster with region-skewed Zipf traffic demonstrates that DEADLINE_DCCC achieves a 54.5% deadline satisfaction rate with 41.4% edge hits, 13.1% cooperative hits, and an average latency of 87.1 ms. While raw hit rates are lower than LRU/LFU baselines, this reflects a deliberate design choice to count only deadline-satisfying hits, making the metric directly meaningful for real-time applications.

Future work will explore (i) joint training using centralised critics with decentralised actors (CTDE) to accelerate convergence while retaining runtime independence, (ii) integration with EdgeSimPy for higher-fidelity mobility and energy modelling, (iii) dynamic coefficient adaptation through meta-learning to handle workload shifts, and (iv) validation on real CDN traces with measured deadline distributions.

Acknowledgement

The authors thank the Department of Computer Science and Engineering at Dr. B. R. Ambedkar NIT Jalandhar for providing computational resources and the DCCC research group for helpful discussions.

References

- [1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, Oct. 2016.
- [2] A. Al-Habob, O. Dobre, H. Yanikomeroglu, and S. Muhaidat, "Towards intelligent adaptive edge caching using deep reinforcement learning," *IEEE Transactions on Vehicular Technology*, vol. 71, no. 8, pp. 8983–8998, Aug. 2022.
- [3] A. Sadeghi, F. Sheikholeslami, and G. B. Giannakis, "Optimal and scalable caching for 5G using reinforcement learning of space-time popularities," *IEEE Journal of Selected Topics in Signal Processing*, vol. 12, no. 1, pp. 180–190, Feb. 2019.
- [4] C. Zhong, M. C. Gursoy, and S. Velipasalar, "A deep reinforcement learning-based framework for content caching," in *Proc. 52nd Annual Conference on Information Sciences and Systems (CISS)*, Princeton, NJ, Mar. 2020, pp. 1–6.
- [5] J. Liu, B. Bai, J. Zhang, and K. B. Letaief, "Cache placement in fog-RANs: From centralized to distributed algorithms," *IEEE Transactions on Wireless Communications*, vol. 16, no. 11, pp. 7039–7051, Nov. 2017.
- [6] S. Traverso, M. Ahmed, M. Garetto, P. Giaccone, E. Leonardi, and S. Niccolini, "Temporal locality in today's content caching: Why it matters and how to model it," *ACM SIGCOMM Computer Communication Review*, vol. 43, no. 5, pp. 5–12, Oct. 2013.
- [7] S. Borst, V. Gupta, and A. Walid, "Distributed caching algorithms for content distribution networks," in *Proc. IEEE INFOCOM*, San Diego, CA, Mar. 2010, pp. 1–9.
- [8] N. Golrezaei, K. Shanmugam, A. G. Dimakis, A. F. Molisch, and G. Caire, "FemtoCaching: Wireless content delivery through distributed caching helpers," *IEEE Transactions on Information Theory*, vol. 59, no. 12, pp. 8402–8413, Dec. 2013.
- [9] W. Zheng, L. Gao, G. Cao, and H. Zhang, "Heterogeneous cellular network with cooperative caching: A matching and coalition formation game approach," *IEEE Transactions on Vehicular Technology*, vol. 67, no. 7, pp. 6415–6428, Jul. 2018.

- [10] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial Intelligence*, vol. 101, no. 1–2, pp. 99–134, 1998.
- [11] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Advances in Neural Information Processing Systems (NeurIPS)*, Long Beach, CA, Dec. 2017, pp. 6379–6390.
- [12] J. Xu, L. Chen, and P. Zhou, "Joint service caching and task offloading for mobile edge computing in dense networks," in *Proc. IEEE INFOCOM*, Paris, France, Apr. 2019, pp. 207–215.
- [13] C. Liu, M. Bennis, M. Debbah, and H. V. Poor, "Dynamic task offloading and resource allocation for ultra-reliable low-latency edge computing," *IEEE Transactions on Communications*, vol. 67, no. 6, pp. 4132–4150, Jun. 2019.
- [14] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, Feb. 2015.
- [15] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. New York, NY: Springer, 2011.

Algorithm 1 DEADLINE_DCCC Routing Decision

Require: request (c, u) , edge cluster $\mathcal{N}$, deadlines D_1, D_2 ,

DQN agents $\{\pi_i\}_{i \in \mathcal{N}}$, time step t

Ensure: served latency ℓ , hit type h

```

1:  $n_{\text{local}} \leftarrow$  home node for region  $u$ 
2:  $\mathbf{o} \leftarrow \text{BuildObservation}(n_{\text{local}}, c, t)$ 
3:  $a \leftarrow \pi_{n_{\text{local}}}(\mathbf{o})$  ▷ DQN action
4: if  $a = \text{serve\_local}$  or  $n_{\text{local}}.\text{has}(c)$  then
5:    $\ell \leftarrow \ell_{\text{edge}}(n_{\text{local}}, c, u)$ 
6:   if  $\ell \leq D_1$  then
7:     return  $\ell$ , edge_hit
8:   end if
9: end if
10: if  $a = \text{forward\_dccc}$  or  $a = \text{default\_policy}$  then
11:    $p_c \leftarrow \text{NewtonPopularity}(c, t)$ 
12:   for each node  $n \in \mathcal{N}$  do
13:      $\Omega_n \leftarrow \text{CacheOverlap}(n, \mathcal{N})$ 
14:      $\ell_n \leftarrow \ell_{\text{edge}}(n, c, u)$ 
15:      $\sigma_n \leftarrow \text{DeadlineScore}(n, c, p_c, \ell_n, \Omega_n, D_1)$ 
16:   end for
17:    $n^* \leftarrow \arg \max_{n \in \mathcal{N}} \sigma_n$ 
18:    $\ell \leftarrow \ell_{\text{cooperative}}(n^*)$ 
19:   if  $\ell \leq D_2$  then
20:     if  $n^*.\text{has}(c)$  then
21:       return  $\ell$ , coop_hit
22:     else
23:       ICEEvict( $n^*, c, p_c$ ) ▷ Cache content at
24:       best node
25:       return  $\ell$ , coop_hit
26:     end if
27:   end if
28:  $\ell \leftarrow \ell_{\text{cloud}}(c)$ 
29: return  $\ell$ , cloud_fetch

30: function DeadlineScore( $n, c, p_c, \ell_n, \Omega_n, D_1$ )
31:    $\chi \leftarrow 1.0$  if  $n.\text{has}(c)$  else  $\min(0.5, p_c)$ 
32:    $\delta \leftarrow \max(0, \ell_n - D_1)$ 
33:   return  $2.0p_c + 6.0\chi - 0.5\Omega_n - 0.08\ell_n - 1.2 \cdot n.\text{load}^2 - 0.4\delta$ 
34: end function

35: function ICEEvict( $n, c, p_c$ )
36:   if  $n.\text{cache}$  is full then
37:      $c' \leftarrow \arg \min_{x \in n.\text{cache}} p_x(t) \cdot \log(1 + s_x)$ 
38:     Evict  $c'$  from  $n$ 
39:   end if
40:   Insert  $c$  into  $n.\text{cache}$ 
41: end function

```
